

3-Layer Web Application Architecture

herbert@upi.edu



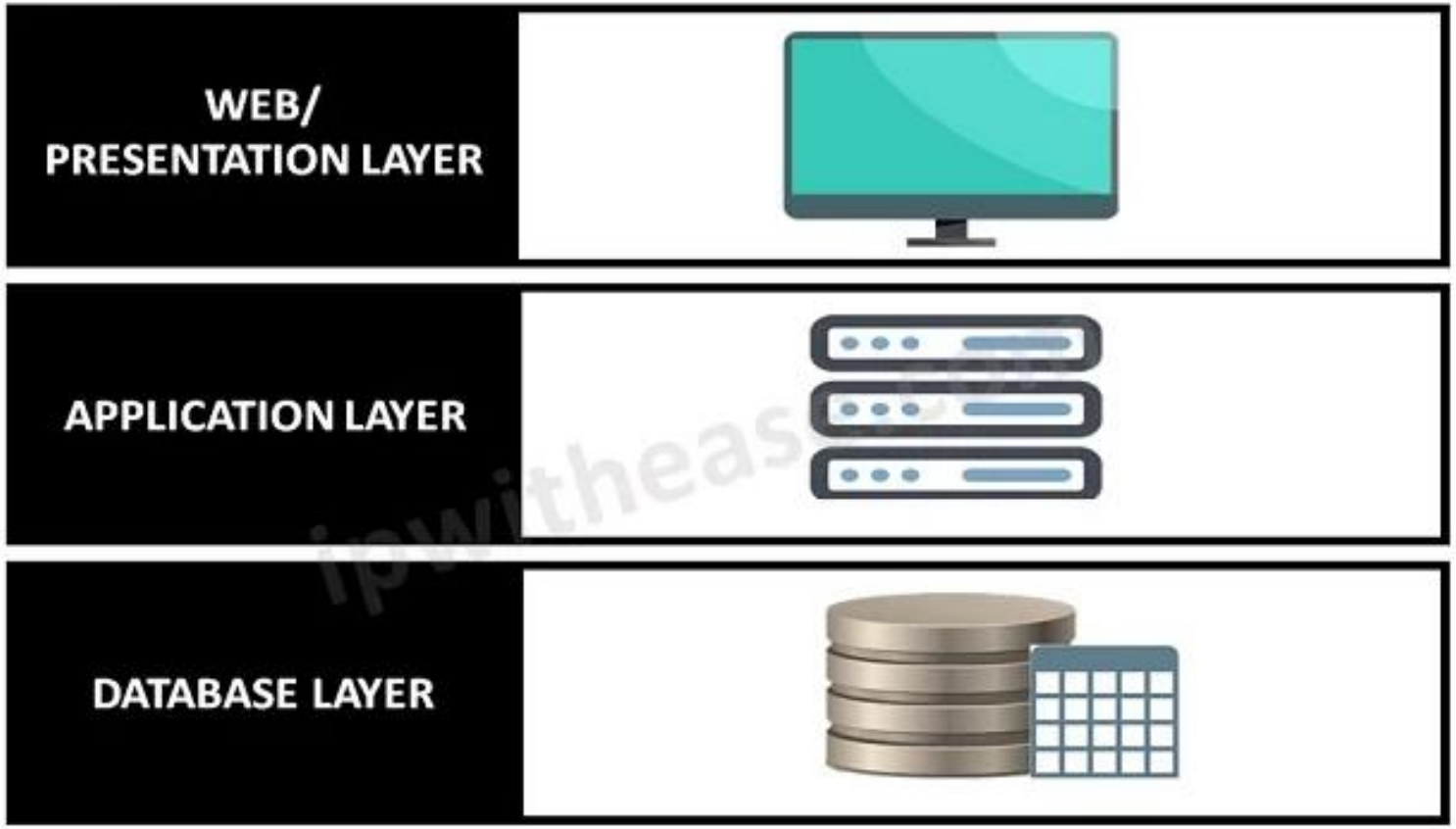
Konsep 3-Layer Web Application

- 3-Layer Web Application Architecture adalah konsep pengembangan aplikasi web dengan membagi sistem menjadi tiga lapisan (layer) yang memiliki tugas dan tanggung jawab berbeda.

- Tujuan utamanya adalah membuat aplikasi lebih terstruktur, mudah dikembangkan, mudah dipelihara, aman, dan fleksibel.

- Tiga lapisan tersebut adalah:
  - Presentation Layer (Lapisan Antarmuka)
  - Application/Business Logic Layer (Lapisan Logika Bisnis)
  - Data Layer (Lapisan Pengelolaan Data)

Ilustrasi Konsep



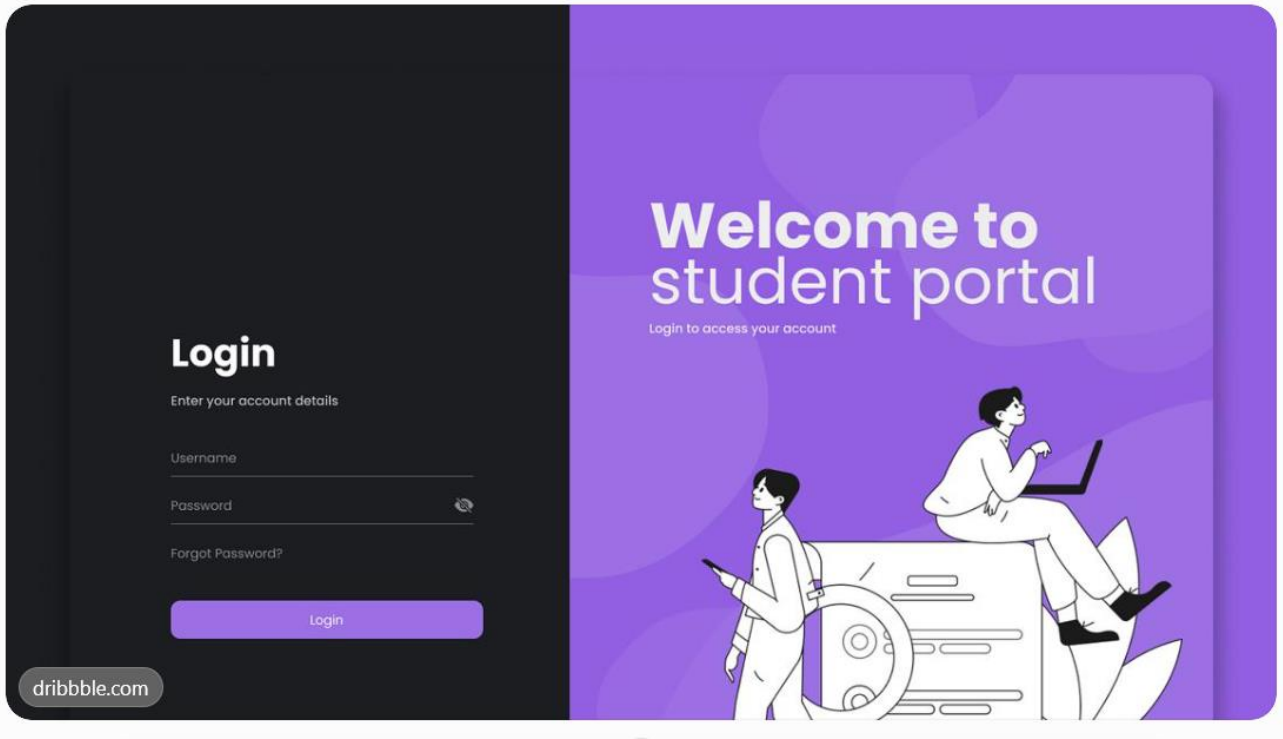


Presentation Layer (Frontend)

- Presentation Layer merupakan bagian aplikasi yang berinteraksi langsung dengan pengguna.
- Lapisan ini bertugas menampilkan informasi dan menerima masukan dari pengguna.
- Teknologi yang digunakan:
  - HTML: membangun struktur halaman.
  - CSS: mengatur tampilan dan desain.
  - JavaScript: membuat halaman interaktif.
  - React, Vue, Angular: framework frontend.

Contoh: Presentation Layer

- Misalnya kita membuat aplikasi Sistem Informasi Mahasiswa



Ketika tombol Login ditekan, Presentation Layer mengirimkan data login ke backend melalui API.

Intinya: Presentation Layer bertanggung jawab terhadap apa yang dilihat dan digunakan oleh pengguna.



Application / Business Logic Layer (Backend)

- Layer ini merupakan bagian yang menangani proses bisnis atau aturan utama aplikasi.
- Lapisan ini menerima permintaan dari frontend, memprosesnya, kemudian berkomunikasi dengan database jika diperlukan.
- Teknologi yang digunakan:
  - PHP (Laravel)
  - Python (Django, Flask)
  - Java (Spring Boot)
  - JavaScript (Node.js, Express)
  - C# (.NET)

Contoh: Application / Business Logic Layer

Contoh kasus login:

Ketika pengguna mengirimkan username dan password, backend akan:

1. Menerima data login dari frontend.
2. Memeriksa apakah username tersedia.
3. Memverifikasi password menggunakan mekanisme hashing.
4. Menentukan apakah pengguna berhak masuk.
5. Mengirimkan hasil autentikasi ke frontend.

```php
$username = $_POST['username'];
$password = $_POST['password'];

// Mencari mahasiswa berdasarkan username
$user = findUser($username);

if ($user && password_verify(
    $password,
    $user['password']
)) {
    echo "Login berhasil";
} else {
    echo "Username atau password salah";
}

?>
```

Intinya: Business Logic Layer bertanggung jawab terhadap bagaimana aplikasi bekerja dan mengambil keputusan.



Data Layer (Database)

- Data Layer bertugas menyimpan, mengambil, memperbarui, dan menghapus data.

- Lapisan ini biasanya berisi database dan mekanisme akses data.

- Teknologi yang digunakan:
  - MySQL
  - PostgreSQL
  - Microsoft SQL Server
  - Oracle Database
  - MongoDB

Contoh: Data Layer

Tabel mahasiswa

id	nama	nim	prodi
1	Andi	23001	Informatika
2	Budi	23002	Sistem Informasi
3	Citra	23003	Informatika

Tabel users

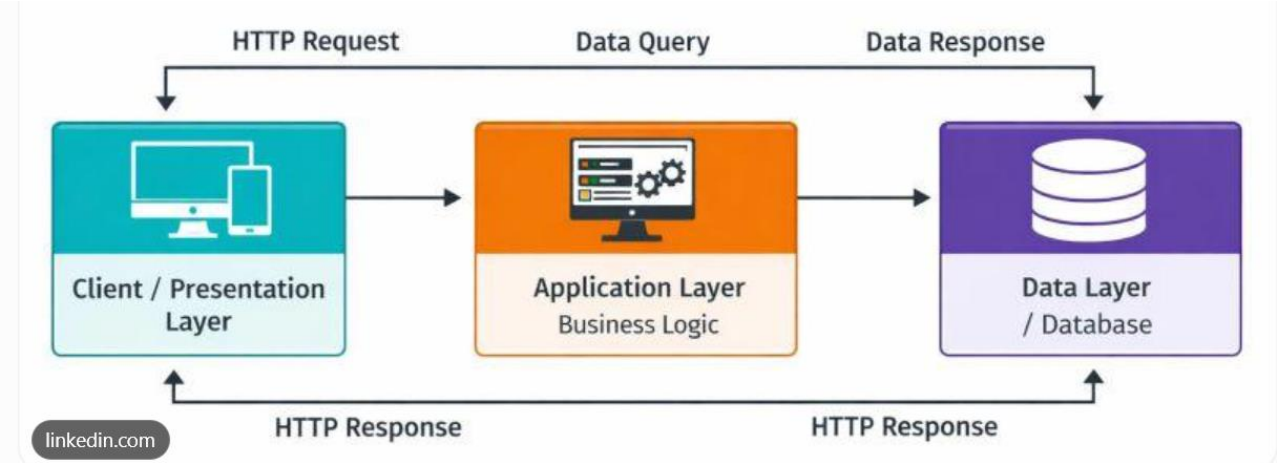
id	username	password
1	andi01	Hash password
2	budi02	Hash password

Ketika backend membutuhkan informasi mahasiswa, backend akan mengirimkan query ke database.

Database kemudian mengembalikan data yang diminta kepada backend.

```
SELECT *
FROM mahasiswa
WHERE nim = '23001';
```

Intinya: Data Layer bertanggung jawab terhadap penyimpanan dan pengelolaan data aplikasi.



Alur prosesnya:

Tahap	Layer	Aktivitas
1	Presentation	Mahasiswa menekan menu KHS
2	Business Logic	Backend menerima permintaan
3	Business Logic	Backend memeriksa autentikasi dan hak akses
4	Data Layer	Backend meminta data nilai ke database
5	Data Layer	Database mengembalikan data nilai
6	Business Logic	Backend mengolah data KHS
7	Presentation	Frontend menampilkan KHS

Tanggung Jawab Ketiga Layer

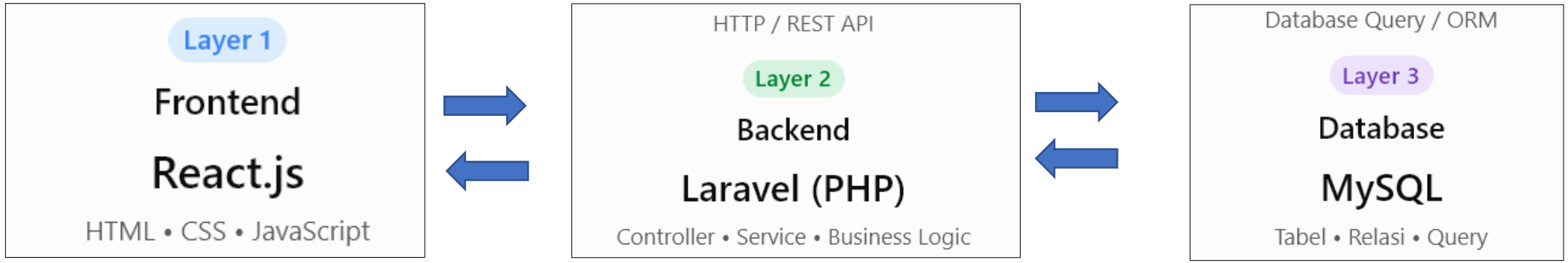
Aspek	Presentation	Business Logic	Data
Fungsi	Tampilan	Proses bisnis	Penyimpanan
Berinteraksi dengan	Pengguna	Frontend dan database	Backend
Contoh	HTML, CSS	PHP, Python	MySQL
Tanggung jawab	Menampilkan data	Mengolah data	Menyimpan data



Mengapa Menggunakan Arsitektur 3-Layer?

1. **Separation of concerns**: Setiap layer mempunyai tanggung jawab masing-masing.
2. **Maintainability**: Perubahan pada tampilan tidak harus mengubah struktur database.
3. **Scalability**: Setiap layer dapat dikembangkan sesuai kebutuhan.
4. **Security**: Database tidak perlu diakses secara langsung oleh pengguna.
5. **Reusability**: Backend API dapat digunakan oleh aplikasi web maupun mobile.

Implementasi Teknologi



Kombinasi tersebut dapat digunakan untuk:

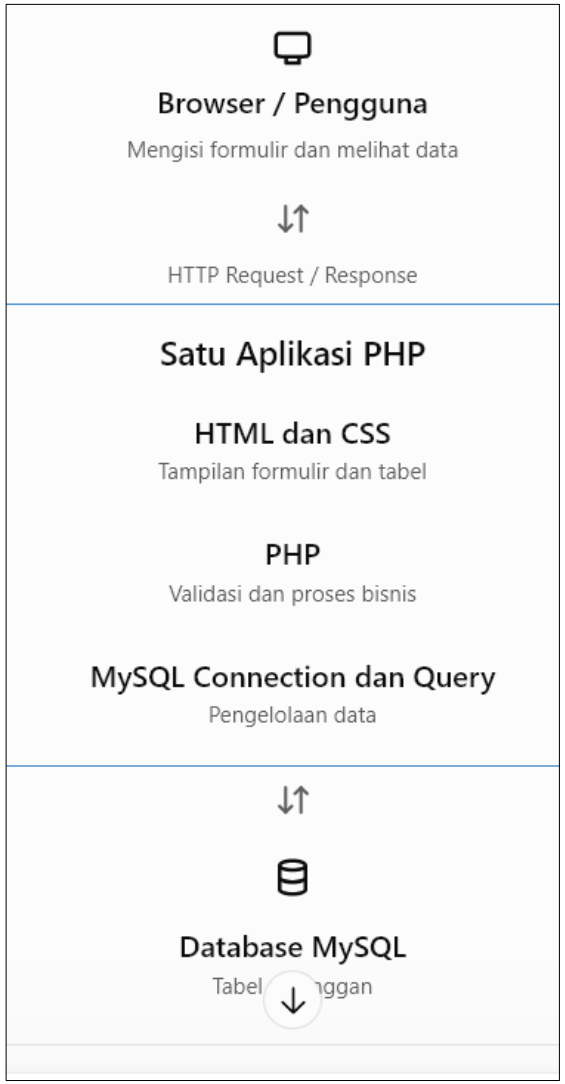
- Sistem Informasi Akademik.
- Sistem Informasi Perpustakaan.
- E-Commerce.
- Sistem Informasi Rumah Sakit.
- Sistem Informasi Kepegawaian.

Konsep Aplikasi Tanpa 3-Layer

- Pendekatan ini biasanya disebut arsitektur **monolitik** sederhana atau monolithic application, dengan kode frontend, proses bisnis, dan akses database yang tidak dipisahkan secara tegas.

- Misalnya membuat aplikasi Data Pelanggan menggunakan PHP Native dan MySQL.

- Seluruh proses dapat ditempatkan dalam satu file PHP, mulai dari:
  - Menampilkan formulir.
  - Menerima input pengguna.
  - Memvalidasi data.
  - Mengakses database.
  - Menampilkan hasil.



Cara Kerja Aplikasi Tanpa 3-Layer?

1. **Pengguna mengisi formulir**

   Nama, telepon, email, dan alamat.

2. **Browser mengirim data ke index.php**

   Menggunakan HTTP POST.

3. **PHP memproses data**

   Melakukan validasi dan menjalankan query SQL.

4. **MySQL menyimpan data**

   Data dimasukkan ke tabel pelanggan.

5. **PHP mengambil data terbaru**

   Menjalankan SELECT.

6. **Browser menampilkan data**

   Pengguna melihat daftar pelanggan terbaru.

Tidak ada pemisahan: frontend, business logic, dan data access secara tegas.

Semua dikelola dalam **satu** aplikasi PHP.

Perbandingan Dengan 3-Layer Architecture

Aspek	Tanpa 3-layer	Dengan 3-layer
Struktur	Sederhana	Terpisah
File program	Bisa satu file	Beberapa komponen
Frontend	Menyatu dengan PHP	Terpisah
Business logic	Menyatu dengan kode aplikasi	Dipisahkan
Database access	Langsung dari file PHP	Melalui komponen data access
Pemeliharaan	Lebih sulit jika kompleks	Lebih terstruktur
Pengembangan	Cepat untuk aplikasi kecil	Lebih terorganisasi
Pengujian	Cenderung lebih sulit	Lebih mudah dipisahkan

Catatan:
Aplikasi satu file ini bukan berarti sama sekali tidak memiliki tiga tanggung jawab tersebut.

Fungsi tampilan, proses bisnis, dan akses data tetap ada, tetapi tidak dipisahkan menjadi layer yang jelas.



Pertanyaan?

- Mana yang lebih mudah dibuat?
- Mana yang lebih mudah diperbaiki ketika terjadi kesalahan?
- Mana yang lebih mudah dikembangkan ketika fitur bertambah?
- Apa risiko jika seluruh kode ditempatkan dalam satu file?
- Apakah aplikasi tanpa 3-layer selalu lebih buruk?

Tugas Kelompok

- Analisis dan Implementasi Perbandingan Layered vs Non-Layered Web Application Architecture

- Pendekatan **pertama** adalah Non-Layered Architecture, yaitu aplikasi dikembangkan tanpa pemisahan layer secara tegas. Tampilan, proses bisnis, dan akses database dapat berada dalam satu file atau beberapa file yang belum memiliki pemisahan tanggung jawab yang jelas.

- Pendekatan **kedua** adalah Layered Architecture, yaitu aplikasi dikembangkan dengan memisahkan tanggung jawab menjadi tiga lapisan:
  - Presentation Layer.
  - Business Logic Layer.
  - Data Layer.

Studi kasus: Customer Management System

Setiap kelompok diminta membuat aplikasi web untuk mengelola data pelanggan sebuah perusahaan. Aplikasi harus memiliki fitur:

No.	Fitur	Keterangan
1	Tambah pelanggan	Memasukkan data pelanggan baru
2	Tampil pelanggan	Menampilkan seluruh data pelanggan
3	Edit pelanggan	Mengubah informasi pelanggan
4	Hapus pelanggan	Menghapus data pelanggan
5	Pencarian	Mencari berdasarkan nama atau email
6	Validasi	Memeriksa kelengkapan dan keunikan email



Aplikasi pertama: Non-Layered Architecture

Mahasiswa diminta membangun aplikasi dengan pendekatan sederhana menggunakan PHP Native dan MySQL.

Ketentuan:

- Tampilan HTML dapat digabungkan dengan PHP.
- Proses bisnis dapat ditempatkan dalam file yang sama dengan tampilan.
- Koneksi database dan query dapat ditulis langsung dalam file aplikasi.
- Tidak diwajibkan menggunakan REST API.
- Seluruh fitur CRUD harus berjalan.

Aplikasi kedua: 3-Layer Architecture

Mahasiswa diminta membangun kembali aplikasi yang sama dengan memisahkan tanggung jawab setiap layer.

Ketentuan:

- Presentation Layer menangani tampilan dan interaksi pengguna.
- Business Logic Layer menangani proses dan aturan bisnis.
- Data Layer menangani komunikasi dengan database.Komunikasi frontend dan backend menggunakan REST API dan JSON.
- Gunakan PHP native alias non framework.
- Seluruh fitur CRUD harus berjalan.

Luaran Yang Harus Dikumpulkan & Dipresentasikan

- Source code aplikasi Non-Layered
- Source code aplikasi 3-Layer Architecture
- File database SQL
- Diagram arsitektur kedua aplikasi
- Screenshot tampilan kedua aplikasi
- Tabel hasil pengujian dan perbandingan
- Laporan analisis (PDF)
- Slide presentasi kelompok



Terima Kasih